Tsinghua Science and Technology
Doi: xxxx

Research Article

Hybrid Workload-Aware Graph Partitioning Using Streaming Partitioning and Refinement

Vladimir M. Kozlov

© The Author(s)

Abstract Efficient graph partitioning is a critical factor in achieving high performance and scalability in distributed graph databases. Conventional graph partitioning methods, such as streaming graph partitioning and METIS, rely exclusively on the structural properties of the graph and do not account for graph access patterns or query workloads. As a result, these methods primarily optimize partition quality by minimizing the number of inter-storage edges (cut size). However, edges crossing storage boundaries may be frequently accessed during query execution, leading to increased communication overhead and degraded query performance. This paper presents a comparative analysis of workload-aware graph partitioning methods and their integration with streaming graph partitioning. The evaluated approaches are compared in terms of partition quality and their impact on query execution performance in distributed graph databases.

Keywords graph partitioning, distributed graph databases, streaming graph partitioning, Fennel, Kernighan–Lin algorithm, SCHISM, WAWPart, graph algorithms, workload aware partitioning

1 Introduction

The increasing volume and complexity of graph-structured data have led to the widespread adoption of distributed graph databases in domains such as social networks, recommendation systems, knowledge graphs, and cybersecurity. By distributing graph data across multiple storage nodes, these systems can scale to billions of vertices and edges while supporting parallel query execution. However, the efficiency of a distributed graph database largely depends on the quality of graph partitioning. Poor partitioning increases the amount of inter-node communication during query execution, leading to higher latency and reduced overall system performance.

Traditional graph partitioning algorithms, including METIS [**?**], Kernighan–Lin, and more recent streaming approaches such as Fennel [**?**], primarily optimize graph structure by minimizing the number of edges crossing partition boundaries while maintaining balanced partition sizes. Although minimizing the cut size is an important objective, it does not necessarily correspond to improved query performance. In many real-world applications, only a subset of graph edges is frequently traversed. Consequently, even a partitioning with a relatively small cut size may result in substantial communication overhead if frequently accessed edges are distributed across different partitions. An example of this situation is shown in Fig. 1, where the colored edges represent the most frequently traversed edges. The partitioning on the left minimizes the edge cut; however, many of the cut edges are heavily traversed during query execution, resulting in a large number of Inter-Partition Traversals (IPTs). In contrast, the partitioning on the right has a larger edge cut but significantly fewer IPTs.

To address this limitation, several workload-aware partitioning techniques have been proposed. These methods incorporate information about the actual query workload into the partitioning process, seeking to reduce the number of inter-partition traversals during query execution. Examples include Schism [**?**] and WAWPart [**?**], which optimize data placement according to expected access patterns rather than relying solely on graph topology.

This paper investigates whether combining structural and workload-aware partitioning techniques can improve partition quality from the perspective of query execution. Specifically, we evaluate combinations of the Fennel streaming partitioning algorithm with the Kernighan–Lin refinement algorithm, Schism, and WAWPart. The evaluated approaches are compared using the Inter-Partition Traversal (IPT) metric, which measures the number of graph traversals crossing partition boundaries during query execution. Unlike traditional graph cut metrics, IPT directly reflects the communication overhead experienced by distributed graph databases. Furthermore, unlike execution time, IPT is independent of the implementation details and performance characteristics of a particular graph database system, making it a more objective metric for

Manuscript received: 2026-06-14; revised: 2026-06-xx; accepted: 2026-06-xx

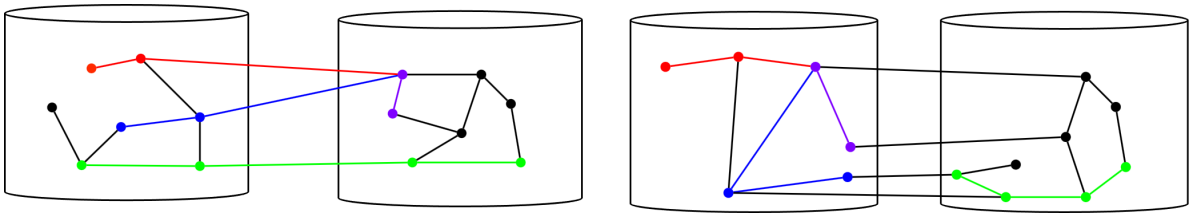

Fig. 1 Example illustrating that a minimum edge cut does not necessarily minimize inter-partition traversals. Colored edges represent the most frequently traversed edges.

evaluating partitioning quality.

This paper presents a comparative evaluation of hybrid graph partitioning strategies that combine the Fennel streaming partitioning algorithm with refinement and workload-aware optimization techniques, including Kernighan–Lin, Schism, and WAWPart.

2 Proposed Hybrid Partitioning Method

One of the methods reviewed in this paper is Schism [**?**]. Without considering implementation details, the core idea of Schism is to assign a special weight to each edge, referred to as the request weight (r-weight). The r-weight of an edge is increased every time the corresponding edge is traversed during query execution. As a result, frequently accessed edges gradually obtain higher weights. Instead of minimizing the number of inter-storage edges or the size of the edge cut, Schism aims to minimize the total weight of the cut. Consequently, edges with high r-weights, representing frequently accessed relationships, are more likely to be placed within the same storage node, while rarely accessed edges are allowed to remain between partitions.

Another workload-aware partitioning method considered in this paper is WAWPart [**?**]. Although the original work neither explicitly discusses Schism nor compares it with WAWPart, both methods are based on the same fundamental concept: assigning r-weights to edges and increasing these weights according to query usage. The main difference is that WAWPart does not increase the weights of all traversed edges equally. Instead, the increase depends on the traversal order, with edges encountered earlier receiving larger weight increments and later edges receiving smaller increments. The partitioning objective remains the same: minimizing the total weight of the cut.

The similarity between Schism and WAWPart raises the question of their relative effectiveness and whether workload-aware optimization provides significant advantages over traditional minimum-cut partitioning approaches.

To exploit the optimization principles of the previously described workload-aware methods, an algorithm capable of minimizing weighted edge cuts is required. A suitable candidate is METIS [**?**], a state-of-the-art graph partitioning framework.

METIS extensively employs the Kernighan–Lin (KL) refinement algorithm, which evaluates the benefit of moving a vertex to another partition using the following gain function:

$$g_v = \sum_{(v,u)\in E \wedge P[v]\neq P[u]} w(v,u) - \sum_{(v,u)\in E \wedge P[v]=P[u]} w(v,u).$$

This heuristic represents the difference between the total weight of edges connecting a vertex to external partitions and the total weight of edges connecting it to its current partition. Consequently, KL naturally attempts to minimize the weighted edge cut and can therefore be applied as a refinement stage after workload information has been collected.

However, METIS has relatively high computational overhead due to its multilevel coarsening, initial partitioning, and uncoarsening phases. Therefore, in this work, these phases are replaced by the streaming graph partitioning algorithm Fennel. Due to the nature of streaming partitioning, Fennel alone cannot perform workload-aware partitioning because vertices and edges are assigned during graph loading before their future access patterns are known. Workload-aware behavior can only be introduced if additional semantic information about vertices is available or if the graph is reloaded together with workload-derived information. Such approaches are beyond the scope of this paper.

Our preliminary experiments indicate that Fennel produces sufficiently high-quality initial partitions, allowing KL-based refinement to converge in a relatively small number of iterations.

Accordingly, the proposed partitioning process consists of three phases:

(1) Initial graph loading, during which vertices are assigned to partitions using either Fennel or random partitioning;
(2) Workload execution, during which edge access statistics are collected;
(3) Iterative partition refinement using Schism, WAWPart, or no refinement.

3 Experiments data generation

The experiments were conducted using synthetically generated random geometric graphs. Graphs containing 5 000, 10 000, 20 000, 50 000, 100 000, and 250 000 vertices were generated. For each graph size, the average vertex degree was fixed at 7, ensuring comparable graph density while varying only the number of vertices. Each graph was partitioned into 4, 6, 8, 10, 12, and 16 partitions, corresponding to the number of storage nodes.

The workload consisted of path-finding queries, each represented by a pair of source and destination vertices. To approximate realistic graph access patterns, query pairs were generated according to the following distribution:

(1) uniformly random source–destination pairs (40%);
(2) random vertex pairs whose Euclidean distance does not exceed a predefined threshold defined for each graph individually (30%);
(3) random pairs in which one vertex is selected from a predefined set of cluster centers (20%);
(4) random pairs with both vertices belonging to the same cluster (5%);
(5) random pairs whose shortest-path distance is equal to a predefined number of hops (5%).

For each graph, a workload consisting of 10 000 queries was generated.

4 Experiments

The primary evaluation metric was the Inter-Partition Traversal (IPT) percentage, defined as the percentage of edges in the computed query paths that crossed partition boundaries. Paths requiring an excessive number of inter-partition traversals were considered infeasible and were therefore excluded from the evaluation. For graphs partitioned using Fennel, the proportion of excluded queries was negligible (approximately 5%). In contrast, for randomly partitioned graphs, the proportion of excluded queries occasionally reached 80%, illustrating the poor quality of purely random partitioning.

To ensure fair comparisons, all random partitionings were generated using a fixed random seed. Consequently, each graph had the same initial random partitioning across all experiments.

Each graph was processed as follows. First, vertices were assigned to partitions either using the Fennel streaming partitioning algorithm or random partitioning, depending on the experimental configuration. Next, an initial refinement step was performed when applicable. The query workload was then executed. After every 1 000 processed queries, the current partitioning was refined using the Kernighan–Lin (KL) algorithm together with the r-weight adjustments defined by the selected workload-aware method. This refinement interval was selected as a compromise between partition quality and computational overhead.

Figure 2 presents the results obtained with random initial partitioning. WAWPart achieves the lowest average IPT in most experimental configurations, while Schism outperforms it in couple of individual cases.

Figure 3 presents the results obtained with Fennel as the initial partitioning algorithm. In this configuration, WAWPart consistently produces the highest IPT values and, in all cases, performs worse than applying no refinement at all. This behavior may be explained by the query generation strategy used in the experiments, which does not emphasize the path-prefix locality that WAWPart is designed to exploit. In contrast, Schism generally achieves the lowest IPT values, although in a few configurations its performance is comparable to or slightly worse than using no refinement. The results also indicate that for graphs partitioned into four storage nodes, workload-aware refinement has only a limited effect on IPT, suggesting that Fennel alone already provides sufficiently high partition quality in this configuration.

The experimental evaluation was conducted exclusively on synthetic graphs of moderate size using a predefined query workload distribution which limits the external validity of the obtained results.. Evaluating the proposed approach on real-world graph datasets, realistic query workloads, and larger graphs remains an important direction for future research.

Overall, the experimental results indicate that the combination of Fennel, Schism, and Kernighan–Lin refinement consistently provides the lowest IPT values across the evaluated workloads. These findings suggest that this combination is a promising partitioning strategy for distributed graph databases.

5 Conclusion

This paper investigated the combination of workload-aware graph partitioning techniques with streaming graph partitioning for distributed graph databases. In contrast to conventional partitioning approaches that optimize only graph topology, the proposed hybrid approach combines the scalability of the Fennel streaming partitioning algorithm with workload-aware refinement based on the Kernighan–Lin algorithm using the weighting strategies of Schism and WAWPart.

The evaluated approaches were compared using the Inter-Partition Traversal (IPT) metric, which directly reflects communication overhead during distributed query execution while remaining independent of the implementation details of a

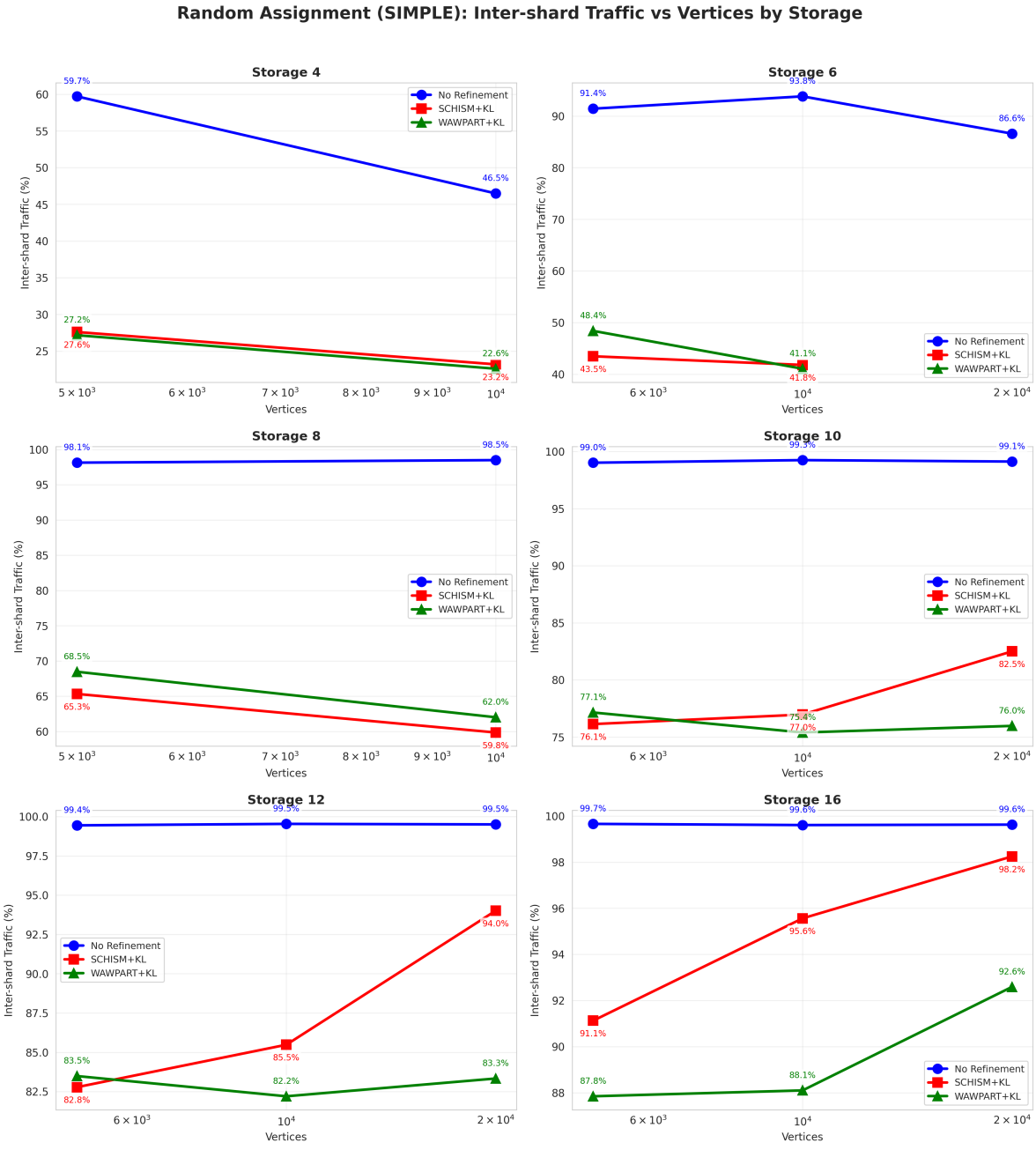

Fig. 2 Experimental results for random initial partitioning.

particular graph database system. Experimental evaluation on synthetic random geometric graphs demonstrated that the combination of Fennel and Schism consistently achieved the lowest IPT values across most experimental configurations. In contrast, WAWPart produced superior results only when the initial partitioning was random and generally did not improve partition quality when applied after Fennel. These results suggest that the effectiveness of workload-aware refinement depends strongly on the quality of the initial partitioning and the characteristics of the query workload.

Overall, the obtained results indicate that combining streaming graph partitioning with lightweight workload-aware refinement is a practical approach for improving partition quality while avoiding the computational overhead of repeated global repartitioning.

Acknowledgments

// TODO:

Declaration of Competing Interest

// TODO:

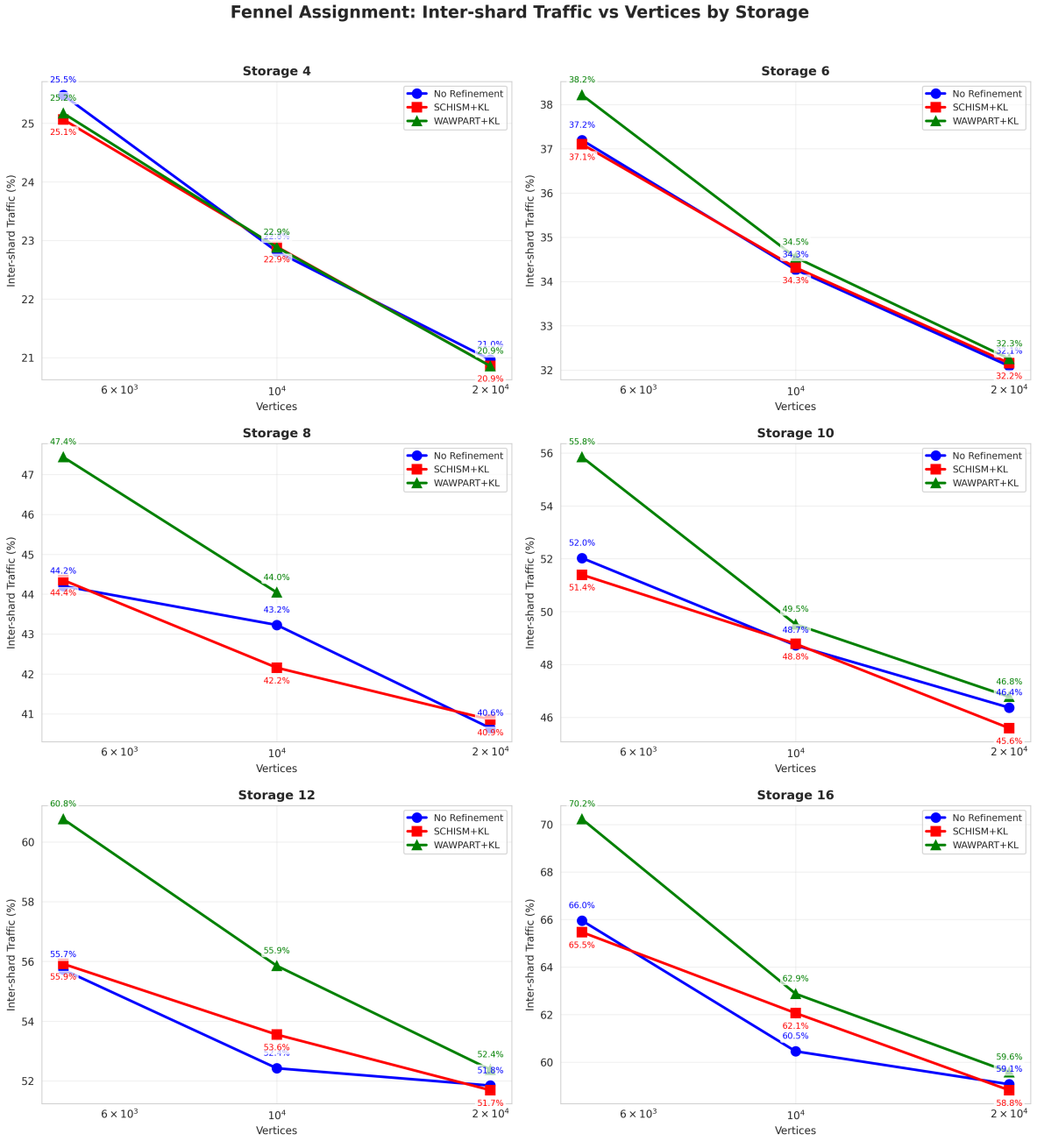

Fig. 3 Experimental results for Fennel initial partitioning.